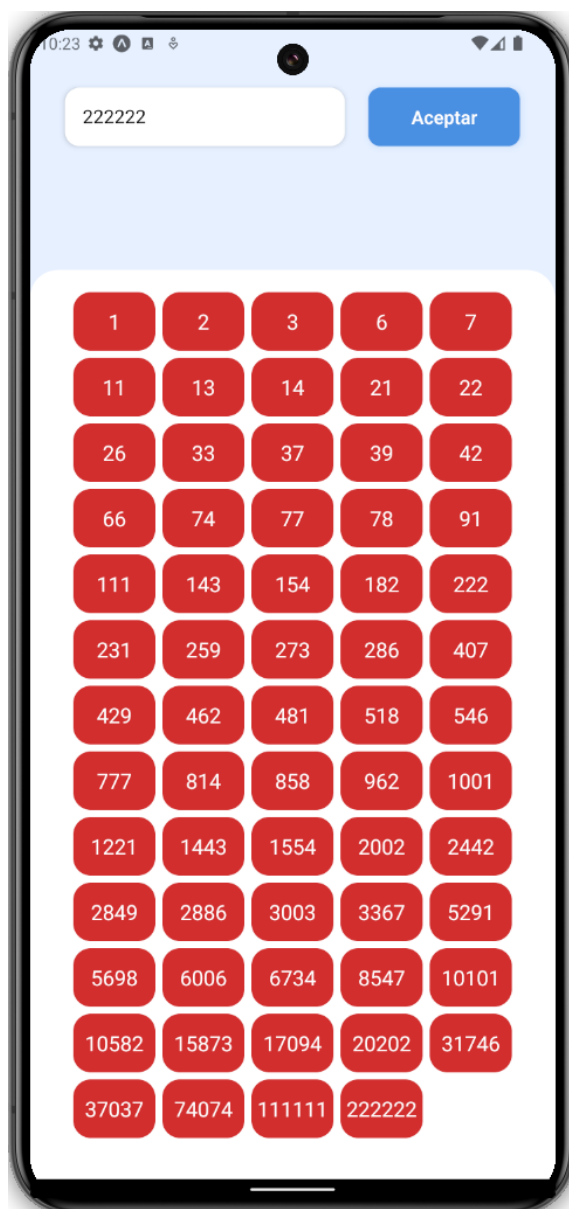


TUTORIAL 13

FUNCIONES ASÍNCRONAS



En este tutorial nos centraremos en la función que calcula los divisores del número introducido. Veremos que esa función es **síncrona**, pero como su resultado tarda un tiempo apreciable en obtenerse (sobre todo cuando se introducen números grandes), vamos a cambiarla para que sea una función **asíncrona**, que devuelva su resultado en el futuro, y adaptaremos la app para ello.

1.- Inicio del proyecto

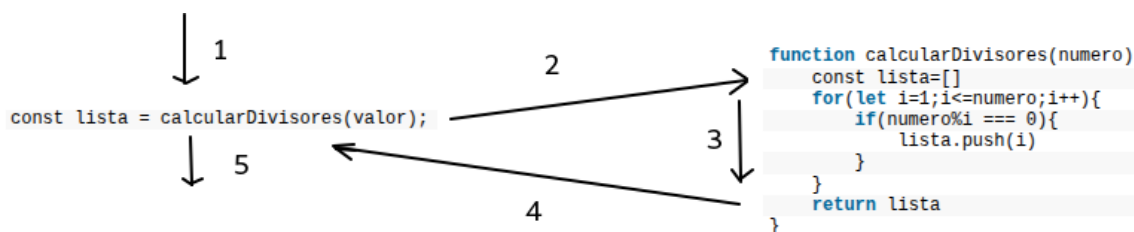
Este proyecto parte del tutorial anterior, así que comenzaremos en el punto donde se quedó

1. Abre el proyecto **rn_divisores** con Visual Studio Code
2. Crea una rama llamada **tutorial13**

2.- Funciones síncronas y asíncronas

Las funciones tradicionales funcionan de manera **síncrona**. Esto quiere decir que la app que las llama, debe esperar a que terminen y así poder acceder al resultado que devuelven con la palabra **return**.

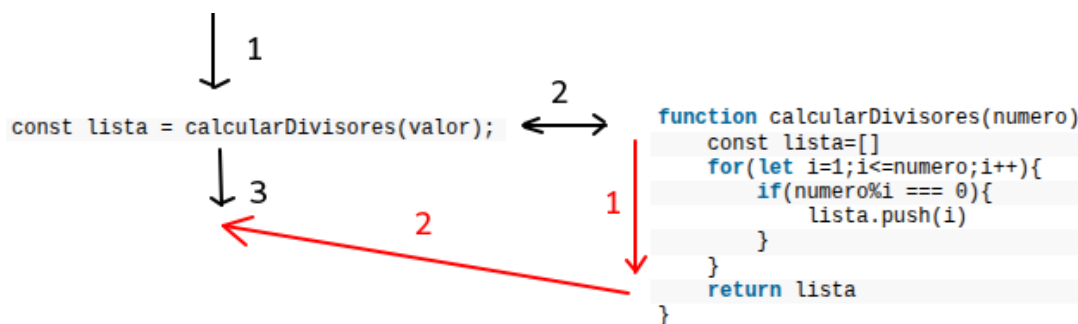
Eso es lo que ocurre con nuestra función **calcularDivisores**, cuyo flujo de ejecución es:



Como puede verse, la función **App** detiene su ejecución hasta que la función **calcularDivisores** le ofrece, vía **return**, la lista de divisores. Estamos viendo una llamada a una **función síncrona**.

Las funciones síncronas son útiles para tareas cortas que terminan rápido, pero hay casos en las que la llamada a la función síncrona tarda un tiempo considerable en realizarse, y eso se nota en la interfaz de usuario (durante el tiempo que tarda en verse la lista de divisores cuando introducimos un número grande la interfaz no responde o parece bloqueada).

Es más adecuado plantear este tipo de tareas para que sean **funciones asíncronas**, que no devuelven su resultado al instante, sino que lo hacen en el futuro. Así, el programa que las llama no tiene que quedarse esperándolo, sino que lo procesará cuando esté disponible.



En este tutorial vamos a convertir **calcularDivisores** en una función asíncrona, y veremos las distintas formas de llamarla desde **App**. Para ello usaremos la clase **Promise**

3.- Promises

Para hacer que una función sea asíncrona, basta con que devuelva un objeto de la clase **Promise**.

La clase **Promise** representa una tarea cuyo resultado es devuelto de forma asíncrona.

- Posee un constructor que recibe una función (o lambda) que realiza la tarea
- Dicha función (o lambda) reciben como parámetros dos funciones llamadas **resolve** y **reject** que sirven para comunicar de forma asíncrona el resultado, o un error, respectivamente

Por tanto, para hacer que **calcularDivisores** sea asíncrona, bastará con que devuelva un objeto **Promise** y se le pasará como parámetro una lambda que calcule los divisores del número. Al finalizar, se usará **resolve** para indicar que la lista obtenida es el resultado que se comunicará asíncronamente.

1. Abre **Funciones.tsx** y modifica la función **calcularDivisores** para que devuelva un **Promise<Array<number>>**, de esta forma:

```
1. function calcularDivisores(numero:number):Promise<Array<number>>{
2.   return new Promise( (resolve,reject) => {
3.     /* aquí calculamos la lista de divisores
4.        y usamos resolve para devolverla de forma asíncrona.
5.        si se produjese un error, usamos reject para comunicarlo */
6.   })
7. }
```

2. Completa el código del lambda que calculando los divisores y usando **resolve** para comunicar el resultado. En este caso, como nunca se producirá un error, no es necesario usar **reject**

```
1. function calcularDivisores(numero: number): Promise<Array<number>> {
2.   return new Promise((resolve, reject) => {
3.     const lista = []
4.     for (let i = 1; i <= numero; i++) {
5.       if (numero % i === 0) {
6.         lista.push(i);
7.       }
8.     }
9.     resolve(lista)
10.   });
11. }
```

Observa que la función asíncrona devuelve (de forma síncrona) un objeto **Promise**, por lo que el lugar que llame a **calcularDivisores** puede continuar su ejecución de forma inmediata, ya que el cálculo de los divisores se realiza dentro de la **Promise**

4.- Manejo de una Promise con then y catch

Para obtener el resultado de una función asíncrona tenemos varias opciones, y una de ellas es usar de forma encadenada los métodos **then** y **catch** del objeto **Promise** que devuelve la función:

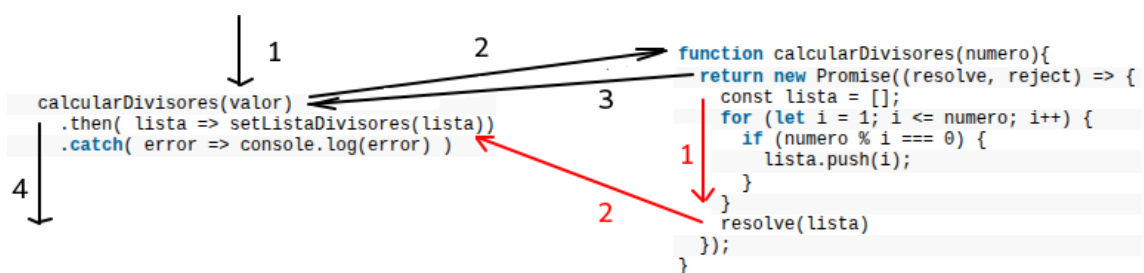
- **then:** Recibe la función (o lambda) que procesará el resultado que la **Promise** devuelve con **resolve**, una vez que esté disponible.
- **catch:** Recibe la función (o lambda) que procesará el error que la **Promise** devuelve con **reject**, en caso de que suceda

1. Abre **App.tsx** y en la función **aceptarPulsado** llama de forma encadenada a **then** y **catch** sobre el objeto **Promise** que devuelve **calcularDivisores**.
 - En **then** pasa un lambda que reciba la lista de divisores y con ella, actualice la variable **listaDivisores**
 - En **catch** pasa un lambda que reciba un error y lo muestre en la terminal (en este caso puede omitirse ya que sabemos que nunca se producirá un error)

```
1. function aceptarPulsado() {  
2.   const { exito, valor } = toEnteroPositivo(texto);  
3.   if (exito) {  
4.     calcularDivisores(valor)  
5.       .then( lista => setListaDivisores(lista))  
6.       .catch( error => console.log(error) )  
7.     setModalVisible(true);  
8.   } else {  
9.     Alert.alert("Error", "Debe introducirse un número entero positivo");  
10.  }  
11. }
```

2. Ejecuta la app y comprueba que todo funciona correctamente.

En este punto hemos realizado una llamada a una **función asíncrona**. El flujo de ejecución es el siguiente:



5.- Manejo de una Promise con async y await

Las funciones asíncronas también pueden ser llamadas de una forma diferente, más parecida a las llamadas síncronas, usando las palabras **async** y **await**

- **async:** Palabra para marcar una función asíncrona (la que devuelve **Promise**)
- **await:** Palabra para esperar el resultado que devuelve una función marcada con **async**. Solo puede ser usada dentro de funciones **async**.

Vamos a reescribir la llamada a **calcularDivisores** para hacerla usando **async/await**, pero antes hay que marcar con **async** la función **calcularDivisores**

1. Abre **Funciones.ts** y marca con la palabra **async** la función **calcularDivisores**

```
1. async function calcularDivisores(numero: number): Promise<Array<number>> {  
2.   // resto omitido  
3. }
```

Una función **async** es asíncrona, y por tanto, siempre debe devolver un **Promise**

2. Abre **App.tsx** y modifica la llamada a **calcularDivisores**, de forma que se llame como si fuera síncrona, pero con la palabra **await**

```
1. function aceptarPulsado() {  
2.   const { exito, valor } = toEnteroPositivo(texto);  
3.   if (exito) {  
4.     const lista = await calcularDivisores(valor)  
5.     setListaDivisores(lista)  
6.     setModalVisible(true);  
7.   } else {  
8.     Alert.alert("Error", "Debe introducirse un número entero positivo");  
9.   }  
10. }
```

La palabra **await** sirve para esperar que esté disponible el resultado que devuelve una función asíncrona, y así poder guardarlo en una variable, como si fuera una llamada síncrona.

3. Observa que al poner **await** se produce un error

```
function aceptarPulsado() {  
  const { exito, valor } = toEnteroPositivo(texto);  
  if (exito) {  
    const lista = await calcularDivisores(valor)  
  }  
}
```

El error se debe a que **await** solo puede ser usado dentro de funciones **async**

4. Convierte **aceptarPulsado** en una función **async**

```
1. async function aceptarPulsado() {  
2.   const { exito, valor } = toEnteroPositivo(texto);  
3.   if (exito) {  
4.     const lista = await calcularDivisores(valor)  
5.     setListaDivisores(lista)  
6.     setModalVisible(true);  
7.   } else {  
8.     Alert.alert("Error", "Debe introducirse un número entero positivo");  
9.   }  
10. }
```

☞ **¿No debería devolver un Promise, por ser función async?** Las funciones marcadas con **async** no necesitan devolver explícitamente un **Promise** y pueden usar **return** para devolver un valor cualquiera. JavaScript se encarga de crear el **Promise**

5. Haz un commit titulado "Finalizado el tutorial 13"
6. Fusiona la rama **tutorial13** con la rama **main**
7. Haz un **push**